Building Agentic Applications with LangChain and Deep Agents

1. Introduction

Purpose: Interactive beginning.

1.1. Discussion

- What are the chatbots you used?
- How long you have been using?
- Have you noticed how these applications changed over last few years?
- What can today's AI assistants do that earlier chatbots couldn't?

Earlier chatbots	Modern AI assistants
Primarily text-based conversations	Multimodal interactions
Respond to individual prompts	Handle multi-step tasks
Limited external capabilities	Can interact with external tools
Mostly stateless interactions	Can maintain conversational context
Generate responses	Can take actions on behalf of users

Can AI perform actions for us instead of just answering?

2. What is an Agent?

Purpose: Introduce to agentic systems.

An agent is an AI system that can reason about a task, decide what actions to take, interact with its environment, and work towards achieving a specified goal.

What makes an application agentic? An application becomes agentic when it gives the AI system the ability to deterimne and execute actions towards a goal.
Examples:

- Reading emails and identifying those requiring attention.
- Writing and executing code, then examining the execution output.
- Searching multiple sources and synthesizing information.
- Interacting with external services through tools and APIs.

The system can observe the results of its actions and use that information to decide what to do next.

3. Chat models and Providers

Purpose: Introduce chat models, model providers and how applications interact with them.

According to what we understood till now, we know we need an intelligent system to do the reasoning and make decisions. We call it the brain of the Agent. Usually we use chat models as the brain of the Agent.

What is the difference between chat models and LLMs?

3.1. Chat Models

There are several Large Language Models (LLMs) available today, such as GPT, Claude, Gemini, Grok, DeepSeek, etc. These models are trained on large amounts of data and can understand and generate human-like text. Chat models are models designed to interact through conversational messages.

Building and training our own LLMs from scratch requires enormous amounts of data, computational resources and infrastructure. This is generally beyond the scope of our application development. Instead, we use already-trained models made available by different providers.

3.2. Model Providers

A model provider is an organization or service that makes models available for applications to use.

Examples:
- OpenAI – GPT models
- Anthropic – Claude models
- Google – Gemini models
- DeepSeek – DeepSeek models
- xAI - Grok models

These providers expose APIs that allow external applications to interact with their models. Each provider may define its own API, authentication mechanism, request format and response format.

For example, when we want to use a model:
1. We obtain an API key from the provider.
2. Our application sends a request to the provider's API.
3. The provider processes the request using the selected model.
4. The model generates a response.
5. The provider returns the response to our application.

Example API references:
- OpenAI API Reference
- Gemini API Reference

Imagine writing an app using OpenAI's api, and later you want to shift to Gemini. Now you need to understand Gemini's API, authentication mechanisms, request and response formats, and we may also need to rewrite code!

What if we had a common interface which deals with different models regardless of the provider API?

This is one of the problems that LangChain helps to solve.

4. What is LangChain

Purpose: Introduce to LangChain

LangChain (github link) is an open-source framework and ecosystem for developing applications powered by Large Language Models (LLMs). It provides abstractions and integrations that simplify working with models, tools, data sources and agentic workflows.

Component	What it provides
LangChain	High-level abstractions for building LLM applications and agents
LangGraph	Graph-based orchestration for stateful and long-running agent workflows
Deep Agents	A higher-level agent harness built on LangGraph
LangSmith	Observability, evaluation and deployment capabilities

5. Foundation

Purpose: Get familiar with initializing a chat model, agent and interacting with them using LangChain.

5.1. Project Setup

Setup `MODEL_PROVIDER`, `MODEL_NAME`, and the corresponding provider API key in the `.env` file.

An `.env.example` file is already provided.

The `main` branch contains two shared modules:

- `settings.py` -> Loads application configuration using Pydantic Settings and loads environment variables using `python-dotenv`.
- `model.py` -> Initializes the chat model used throughout the demo code, except for the first model-connection demo.

5.2. Connecting to a chat model

Demo code: `src/01-foundation/connecting-to-model.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

Try different providers and different models.

For supported providers, changing the provider and model configuration is usually enough. The corresponding LangChain integration must be installed, and LangChain handles the provider-specific interaction underneath.

You will frequently refere to the Documentation of LangChain throughout the session.

Try out parameters such as `temperature`, `max_tokens`, `timeout`, and `max_retries`. Observe how changing these parameters affects the model interaction.

5.3. Creating Agent

Demo code: `src/01-foundation/creating-agent.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.

5.4. System Prompt

Demo code: `src/01-foundation/system-prompt.ipynb`
Running: Execute `uv run jupyter lab` from the repository root and open the notebook.